Project summary

Restaurant Rush is an offline, turn-based restaurant management game in Java with a command-line interface, built and tagged in three runnable stages. The player is the manager: staff time is limited, so every turn they decide which customers and orders come first.

Group	SC2002 SCMA, Group 4
Repository	github.com/DevanshJ07/sc2002-restaurant-rush
Final code	main, commit 1ec828e (11 Nov 2026)
Language and build	Java 17 (--release 17), standard library only, no network or external libraries
Video	https://www.youtube.com/watch?v=y8gwKpnKHsA

Member	Main responsibility
Devansh Jain	Orchestration and integration: GameController, GameCLI, Main, ArrivalSchedule, integration checks, stage tags
Ng Zheng Ning	Domain and staff classes (19 classes) and their domain checks
Braden Lai Xu Kai	Stage 1–3 class diagrams and the sequence diagram
Doan Nhat Minh	Main documentation, SOLID review and the final behaviour-preserving refactor

Tag	Commit	Date	Contents
stage-1	1369fa3	4 Nov 2026	Complete base game with Regular customers, Waiter, Chef, Cashier, both endings
stage-2	9a5d5c6	4 Nov 2026	Adds VIP and Critic customers
stage-3	2ba1f1e	6 Nov 2026	Adds Host, ComboMeal and Happy Hour, with a player-controlled Host and amounts shown in dollars

stage-3-player-host (bc05bb2) is an earlier Stage 3 snapshot kept for reference; stage-3 builds on it. After the tags, main received a refactor that keeps behaviour identical (Section 6).

To build and play from the repository root on Windows: powershell -ExecutionPolicy Bypass -File scripts/run.ps1. To build and run every check: powershell -ExecutionPolicy Bypass -File scripts/build.ps1 -Test (macOS or Linux: sh build-and-check.sh).

1. The game and the rules we defined

The game always offers the brief's base setting, so every run can be reproduced: 18 turns, 2 tables, one Waiter, one Chef and one Cashier (plus one Host in Stage 3), and six customers arriving on turns 1, 4, 7, 10, 13 and 16. The player wins only if, after turn 18, there are at least 4 paid customers, at least $45.00 revenue and an average paid-customer satisfaction of at least 60.

Item	Price	Preparation units	Stage
Salad	$9.00	1	1
Burger	$15.00	1	1
Pasta	$18.00	2	1
Salad + Pasta Combo	$24.30	3	3

Order of one turn (as implemented in GameCLI.run())

1. Status display: turn, revenue, paid count, average, queue, tables, each customer's satisfaction and order, available staff tasks, Happy Hour duration.
2. beginTurn(): staff flags reset, the scheduled customer joins the queue, and Happy Hour recovery runs if it is active.
3. Happy Hour prompt (Stage 3, only until it has been used).
4. Host prompt (Stage 3): seat or wait.
5. Waiter prompt: seat <customerId> <tableId>, order <tableId>, serve <tableId> or wait.
6. Chef prompt: cook <tableId> or wait.
7. finishTurn(): customers who ate for a full turn become ready to pay, the Cashier collects at most one payment, every waiting customer loses satisfaction (leaving at 0), and the Happy Hour timer ticks.

Rules the brief left for us to define

Rule	Our decision	Where it lives
How customers choose food	Regular and VIP take the menu item at index (customerId − 1) mod menu size, so customers 1–6 choose Salad, Burger, Pasta, Combo, Salad, Burger in Stage 3	Customer.chooseItem()
Critic behaviour	Always orders the most expensive item (first on a price tie): Pasta in Stage 2, the Combo in Stage 3	CriticCustomer.chooseItem(), Menu.getMostExpensiveItem()
Stage 2 arrivals	Customer 2 (turn 4) is a VIP and customer 4 (turn 10) is a Critic; the other four stay Regular	ArrivalSchedule.createBaseSchedule()
Cashier "oldest first"	Earliest ready-to-pay turn first; ties go to the lower customer ID	Cashier.collectPayment()
Host	On seat, seats the head of the queue at the	Host.seatNextCustomer()

	lowest-numbered free table; it never uses the Waiter's task	
Who counts as waiting	Anyone queued, or seated and not yet served (including a READY dish not yet served)	Customer.isWaiting()
Happy Hour timing	Offered only before any staff member acts; activation recovers waiting customers at once, and again at the start of its second turn	GameController.activateHappyHour(), beginTurn()
Money	Integer cents throughout, rounded to the nearest cent with Math.round, shown as dollars with two decimals	VIPCustomer, ComboMeal, Order.applyDiscount()
Invalid input	Explains why it failed and asks again; the staff task is not used up	InvalidActionException, GameCLI retry loops

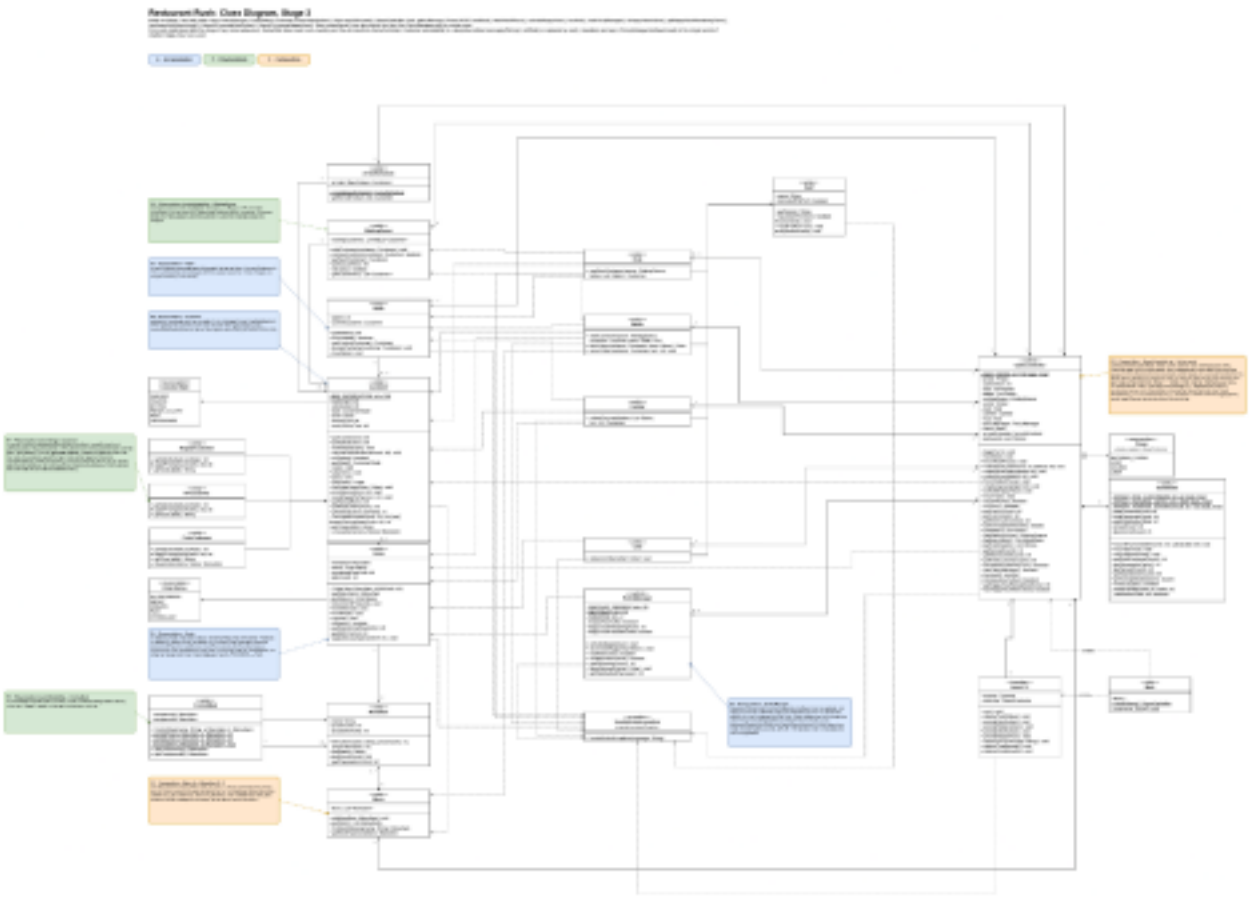
As the brief notes, the customer arriving on turn 16 cannot finish paying within 18 turns, so at most five customers can pay.

2. UML class diagram

Figure 1 is the final class diagram and matches the code on main: 24 source files (25 types, counting the nested Phase enum) in package restaurantrush, split into one boundary class, two control classes, the entities and three enumerations. Stage 1 and Stage 2 diagrams are in the repository for comparison.

Figure 1: Stage 3 class diagram, shown at the top of the next page.

(For proper viewing of the diagrams and figures, please open them from the folders in the GitHub repository or the zip file.)



Classes by role

Role	Classes	Responsibility
Boundary	GameCLI	Reads the manager's commands, calls the controller, prints state; holds no game rules
Control	GameController (with private nested enum Phase)	Runs the turn phases in order, routes staff actions, coordinates arrivals, payment, abandonment and Happy Hour
Control	PolicyManager	Owns the Happy Hour rules: once per game, 2 turns, 20% off, +15 recovery
Entity: customers	abstract Customer; RegularCustomer, VIPCustomer, CriticCustomer; enum CustomerState	Satisfaction, state, food choice and pricing for each customer type
Entity: staff	abstract Staff; Waiter, Chef, Cashier, Host	One task per turn, each staff member's action rules
Entity: orders and menu	Order, enum OrderStatus,	What was ordered, its locked bill and

	Menu, MenuItem, ComboMeal	cooking progress; the menu in order
Entity: restaurant	Table, WaitingQueue, ArrivalSchedule, GameStats	Seating, the queue, the fixed arrivals, and score counting with the win rule
Support	InvalidActionException, Main	Explains a refused action; builds the configuration and starts the game

Relationships and multiplicities

Kind	Relationship	Multiplicity
Inheritance	RegularCustomer, VIPCustomer, CriticCustomer → Customer	–
Inheritance	Waiter, Chef, Cashier, Host → Staff	–
Inheritance	ComboMeal → MenuItem; InvalidActionException → java.lang.Exception	–
Composition	GameController ◆ Table	1 to 1..*
Composition	GameController ◆ Waiter, Chef, Cashier, WaitingQueue, Menu, ArrivalSchedule, GameStats	1 to 1 each
Composition	GameController ◆ Host, PolicyManager	1 to 0..1 (absent in the core-only constructor)
Composition	Menu ◆ MenuItem	1 to 1..*
Association	Customer → Order	1 to 0..1
Association	Order → MenuItem	* to 1
Association	Table → Customer	1 to 0..1
Association	WaitingQueue → Customer	1 to *
Association	ComboMeal → MenuItem (componentA, componentB)	1 to 2
Association	GameCLI → GameController	1 to 1
Dependency	Waiter, Chef, Cashier, Host on Customer, Order, Table, WaitingQueue, Menu; staff, Order and PolicyManager on InvalidActionException	–

OOP annotations on the diagram

- E1 Encapsulation, Order: all state is private with no setters for status or bill. Status moves only through decrementPrepUnit(), markServed(), markPaid() and cancel(), so an order can never jump from IN_PROGRESS to PAID. applyDiscount() works only while IN_PROGRESS.
- E2 Encapsulation, PolicyManager: the timer and “used” flag are private, and “active” is derived from the timer rather than stored, so the Happy Hour rules live in one place and

cannot be bypassed.

- E3 Encapsulation, Table: isOccupied() is derived from currentCustomer != null, so occupancy can never disagree with who is seated.
- E4 Encapsulation, Customer: satisfaction has no setter; it changes only through loseSatisfaction() (own loss rate, never below 0) and recoverSatisfaction() (never above 100). State changes only through seat(), abandon(), pay(), markServed() and markReadyToPay(), each checking that the step is allowed.
- P1 Polymorphism (overriding), Customer: subtypes override getSatisfactionLossRate(), applyPricing() and getTypeLabel(); CriticCustomer also overrides chooseItem(). calculateBill() is a final template method that checks the price once, then calls the subtype's applyPricing().
- P2 Polymorphism (substitutability), WaitingQueue: the queue, Waiter.seatCustomer() and Table.assignCustomer() accept any Customer, so all three types share one service flow.
- P3 Polymorphism (substitutability), ComboMeal: a combo is a MenuItem, so menu, ordering, cooking and billing needed no change in Stage 3.
- C1 Composition, Menu ◆ MenuItem: the menu owns its private item list; others only get a read-only view.
- C2 Composition, GameController ◆ its parts: Main.createGame() builds each part once and passes it to the constructor, which keeps it in a private final field for the whole game; GameStats is created by the controller itself.

3. UML sequence diagram

Figure 2 follows one complete Stage 3 game in which a single manager choice decides the result: switching Happy Hour on at the start of turn 2 records the first customer at satisfaction 100 instead of 92, which turns DEFEAT into VICTORY. The focal object is c1, Regular customer #1, who orders Salad because chooseItem() takes menu index (1 − 1) mod 4 = 0.

Figure 2: Sequence diagram (SC2002_Sequence_Diagram.drawio), shown in three parts as Figures 2a–2c on the pages after the guards and loops list below.

(For proper viewing of the diagrams and figures, please open them from the folders in the GitHub repository or the zip file.)

Objects (lifelines): Manager (player), :GameCLI, :GameController, :PolicyManager, :ArrivalSchedule, :WaitingQueue, :Host, :Waiter, :Chef, :Cashier, table1:Table, c1:RegularCustomer, :Order, :GameStats.

Where each required element appears

Required by the brief	What the diagram shows	Messages
Customer arrival	getArrivalFor(1) returns c1; addCustomer(c1) puts it in the queue	10–11
Host and Waiter assignment	Host seat: seatNextCustomer() picks the lowest free table and calls c1.seat(). Waiter order 1: takeOrder()	20–53

	→ chooseItem(menu) → calculateBill(900) → new Order(Salad, 900)	
Chef assignment, then READY	chefAdvance(1) → advanceOrder() → decrementPrepUnit(); status becomes READY	56–66
SERVED	Turn 2: waiterServe(1) → serveOrder(c1, 2) → order.markServed() (READY → SERVED), c1.markServed(2) (EATING)	107–118
Eating	Turn 3: finishTurn() finds c1 EATING with servedTurn 2 < 3 and calls markReadyToPay(3)	142–146
Payment	Turn 4: collectPayment(tables, 4) picks c1 (ready on turn 3 < 4), calls markPaid(), pay(), freeTable(); the controller calls recordPayment(900, 100) on GameStats	167–180a
Result check	After turn 18: isGameOver(), isVictory() → meetsTargets(), then an alt fragment prints VICTORY or DEFEAT	182–186
A Stage 2 or 3 behaviour	Happy Hour: activateHappyHour() and recoverWaitingCustomers() (+15 to c1, capped at 100), recovery again on turn 3, the timer ticking down; VIP pricing for customer #2 on turn 4 (1500 × 90% = 1350)	85–101, 138–140, 152

Guards, loops and alternatives

- loop [while !controller.isGameOver()] wraps every turn.
- loop [for each staff member] resets the one-task flags in beginTurn().
- loop [until valid choice or wait] around each prompt shows input retry. In turn 2 the manager types cook 1 by mistake: advanceOrder() throws InvalidActionException because the order is SERVED, markActed() is not called, and displayError() asks again (121–129).
- opt guards on each staff action, such as [!hasActedThisTurn() and c1 is SEATED with no order], with InvalidActionException as the else case.
- requirePhase(...) checks the turn phase before each controller action, so the Chef can never act before the Waiter.
- alt [Manager answers "yes"] / [else "no"] for Happy Hour in turn 2, with nested opt [!happyHourHasBeenUsed].
- loop [for each table] for EATING → READY_TO_PAY, and the Cashier's loop [oldest READY_TO_PAY with readyToPayTurn < turn, ties by lower id].
- Final alt [isVictory()] / [else] prints the result.

Figure 2a: Sequence diagram, part 1 of 3: setup and turn 1

Full-resolution image: zoom in to read. Source: SC2002_Sequence_Diagram.drawio.

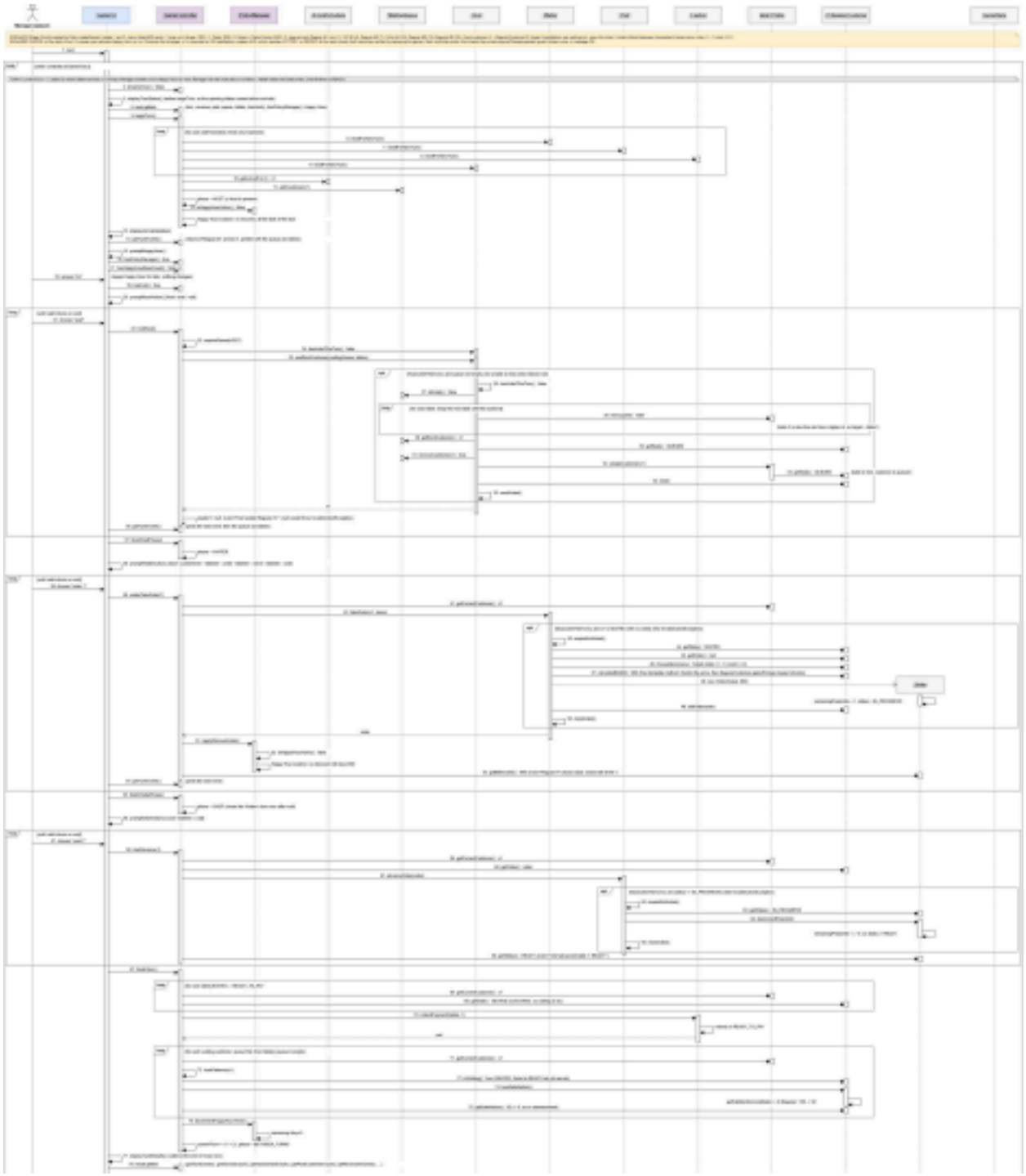


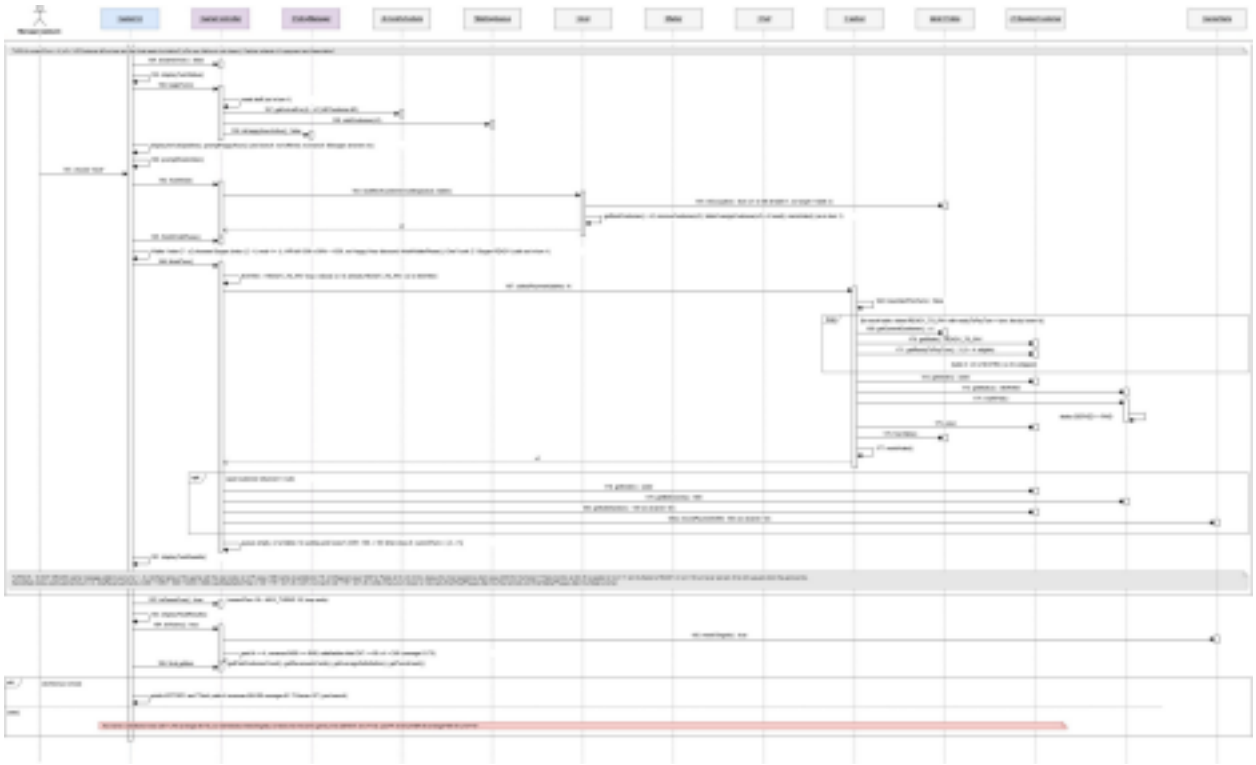
Figure 2b: Sequence diagram, part 2 of 3: turns 2 and 3 (the Happy Hour choice)

Full-resolution image: zoom in to read. Lifeline names repeated from part 1.



Figure 2c: Sequence diagram, part 3 of 3: turn 4 and the result check

Full-resolution image: zoom in to read. Lifeline names repeated from part 1.



How the choice changes the outcome

Turns 5–18 follow the same message patterns and are summarised on the diagram from a replay of the real code. Four customers pay in both branches, for $64.80 in revenue. The only difference is c1's recorded satisfaction:

Turn 2 choice	c1 recorded at	Satisfaction total of 4 paid	Average	Result
Happy Hour on	100	247 (needs ≥ 240)	61.75	VICTORY
Happy Hour off	92	239	59.75	DEFEAT

4. Design considerations

The design puts each game rule in the object that owns the data it needs, keeps the user interface free of rules, and lets new customer types and menu items plug in without changing the turn logic.

Key abstractions and their responsibilities

Abstraction	Responsible for	Deliberately not responsible for
GameCLI	Parsing commands, showing state and errors, re-prompting after a refused action	Any game rule; it only calls controller methods
GameController	Turn phases and their order, routing each command to the right staff member,	Counting score (GameStats), pricing

	arrivals, abandonment, Happy Hour timing	(Customer, PolicyManager), staff limits (Staff)
GameStats	Paid count, revenue, recorded satisfaction, served and abandoned counts, the victory rule	Deciding when a payment happens
Customer and subtypes	Own satisfaction, loss rate, food choice, bill, legal state changes	Deciding when they lose satisfaction (the controller does)
Staff and subtypes	One task per turn; each role's validation and action	Turn order or revenue
Order	Locked bill, cooking progress, legal status transitions	Price rules (computed before it is created)
Menu, MenuItem, ComboMeal	The items in order, prices and preparation units; combo price and units computed once	Who chooses which item
PolicyManager	Happy Hour rules: once per game, 2 turns, 20% discount, +15 recovery	Looping over customers (the controller does)
ArrivalSchedule	Which customer arrives on which turn	Adding them to the queue

Design principles, only where the code shows them

- Single Responsibility. Each class above has one reason to change. The clearest case is the refactor after the stage tags: score counting and the win rule moved out of GameController into GameStats, so the controller now only coordinates turns.
- Open–Closed. Stage 2 added VIPCustomer and CriticCustomer without editing GameController or GameCLI; the stage-1 → stage-2 comparison shows only ArrivalSchedule and Menu changing. Stage 3's ComboMeal is a new MenuItem, so ordering, cooking and billing were untouched. The limit: a new staff role is not closed, because Stage 3 had to add a Host phase to the controller and CLI.
- Liskov Substitution. Every place that takes a Customer (queue, table, Waiter, Cashier) works with any subtype, and every place that takes a MenuItem works with a ComboMeal. calculateBill() is a final template method, so no subtype can skip the price check or weaken it.
- Not claimed: Interface Segregation and Dependency Inversion. The brief asks us not to add interfaces just to satisfy a checklist, and we have none. Objects are built in Main and passed into the controller's constructor, which let our checks build controlled games, but the controller still depends on concrete classes, so this is not DIP.

Trade-offs we accepted

Decision	Benefit	Cost

Typed fields waiter, chef, cashier, host instead of a List<Staff>	No instanceof or casts; each call is type-checked	Adding a role means editing the controller, as the Host did
Optional host and policyManager (null in the core-only constructor)	Stage 1 and 2 behaviour and their checks still run unchanged on the final code	The controller checks for their absence in a few places
Explicit turn phases (Phase, requirePhase())	Actions cannot run out of order; a dish finished by the Chef cannot be served the same turn	More code than a free-form loop
Bill locked in Order when it is taken	Payment just reads the bill; Happy Hour never reprices an old order	VIP and Happy Hour discounts must be applied in the right order at order time
Deterministic food choice instead of randomness	Every run is reproducible, explainable and testable	Less variety between games

Alternative design we considered: an automatic Host. Our refined class design (v3) and first Stage 3 build had the Host seat the next customer automatically at the start of each turn. We replaced it with a player prompt (seat or wait), because an automatic Host removes a decision from the manager, and the brief's focus is on meaningful manager choices. The cost was one more phase in the controller and one more prompt in the CLI.

How the player's choices change the result

- Waiter priority: one task per turn, so choosing to seat, order or serve decides who keeps losing satisfaction.
- Kitchen priority: the Chef advances one unit per turn, so a 3-unit Combo competes with quicker dishes.
- Using the Host: seating with the Host frees the Waiter to take the order in the same turn, saving a turn of waiting.
- Happy Hour timing: it can be used only once. In the sequence diagram scenario, switching it on at turn 2 lifts the satisfaction total from 239 to 247 and turns DEFEAT into VICTORY.
- The two replays: with the same six arrivals, efficient play wins with 5 paid, $73.80 and an 84.80 average, while waiting every turn loses with 0 paid (Section 7).

5. Implementation

All three stages are complete and runnable: the final code is about 1,600 lines of production Java in 24 files, uses only the standard library, and contains no instanceof checks.

Stage 1: the core game

- The full six-step turn from the brief, with explicit phases so the Waiter always acts before the Chef and a dish finished this turn is served next turn at the earliest.
- Eating takes one full turn (markReadyToPay() only when servedTurn < currentTurn), and a

customer who becomes ready to pay pays next turn at the earliest.
- The Cashier collects at most one payment a turn, oldest ready-to-pay turn first, lower customer ID on ties.
- Waiting customers lose satisfaction at turn end. At 0 they leave: any unfinished order is cancelled, the queue place or table is freed, and nothing is added to revenue.
- Victory needs all three targets after turn 18; the game never ends early. The average uses the satisfaction recorded at payment and shows N/A before anyone pays.

Stage 2: customer behaviour

- VIPCustomer: loses 5 per waiting turn; bill Math.round(baseCents × 90 / 100.0), so a Burger costs $13.50.
- CriticCustomer: loses 12 per waiting turn, pays full price, overrides chooseItem() to pick the most expensive item.
- ArrivalSchedule makes customer 2 a VIP and customer 4 a Critic, keeping six arrivals. All types follow the same service flow, dispatched through the Customer methods.

Stage 3: new role and menu policy

- Host seats the head of the queue at the lowest-numbered free table when the manager chooses seat; the Waiter keeps its own task.
- ComboMeal (Salad + Pasta) computes its price once in the constructor: round((900 + 1800) × 0.9) = 2430 cents, with 3 preparation units.
- PolicyManager holds the Happy Hour constants (20%, +15, 2 turns). The controller offers activation only before staff act, applies policyManager.applyDiscount(order) exactly once right after the Waiter takes an order (after the VIP discount: a VIP Burger is $15 × 0.9 × 0.8 = $10.80), and recovers waiting customers at the start of each active turn.
- Order.applyDiscount() works only while the order is IN_PROGRESS, so orders placed earlier keep their locked price, and customers who left are never affected.

Code quality

- Every field is private. Order and Customer have no general setters; state moves only through guarded methods.
- Staff methods validate everything before changing anything and call markActed() last, so a refused action leaves the game unchanged and does not use up the task.
- Expected player mistakes throw the checked InvalidActionException with a reason; programming errors use IllegalArgumentException or IllegalStateException.
- Collections are exposed as read-only views (Collections.unmodifiableList, List.copyOf); magic numbers are named constants (MAX_SATISFACTION, DISCOUNT_PERCENT, the three victory targets in GameStats).
- Money is stored in integer cents everywhere and rounded with Math.round, never truncated.

Reproducibility

- Customer arrivals and food choices are deterministic, so the same inputs always give the same game.
- examples/victory.txt and examples/defeat.txt replay complete games through the real CLI; the README lists the commands and expected results.
- scripts/build.ps1 -Test (Windows) and build-and-check.sh compile with --release 17 and run every check; a failure stops the build with a nonzero exit code.
- Each stage tag can be checked out and run on its own.

6. Development process

We built the stages in order and tagged each only after its real code passed its checks, so every tag is a runnable snapshot. Two coders worked in parallel from a fixed API contract (CODING_PLAN.md): Zheng Ning wrote the domain and staff classes, and Devansh wrote the controller, CLI and setup and integrated each stage. Braden drew the diagrams from the code, and Minh reviewed and refactored the final code and wrote the documentation.

Development log (from the repository history, newest first)

Date (2026)	Who	What
11 Nov	Minh	Pull requests #3–#6 merged: refactor, duplicate-state removal, documentation sync, diagram fixes
10 Nov	Minh	Pull request #2 (clean-up) merged; refactor: template method for billing, GameStats, seat()/abandon()/pay()
9 Nov	Minh	Code-style clean-up, IllegalStateException for internal errors
8 Nov	Braden	Stage 1, 2 and 3 class diagrams and the sequence diagram
6 Nov	Devansh	Player-controlled Host, dollar amounts in messages; stage-3 tagged
4 Nov	Devansh, Zhengning	Stage 1, 2 and 3 integrated with Zhengning's domain code; stage-1 and stage-2 tagged
25 Oct	Devansh	Repository set up; coding responsibilities and API contract agreed

Stage 1 → Stage 2 (compare stage-1...stage-2)

Change	Files	Reason
New	VIPCustomer, CriticCustomer, Stage2Checks, Stage2IntegrationChecks	The two new customer types and their checks
Changed	Menu (+14 lines)	Adds getMostExpensiveItem() for the Critic

Changed	ArrivalSchedule (+2 lines)	Turn 4 becomes a VIP and turn 10 a Critic; still six arrivals
Changed	DomainChecks, README, STAGE_CHANGES.md, examples/victory.txt	Shared check helper; documentation and replay updated for Stage 2
Unchanged	GameController, GameCLI, Main	Evidence that the new behaviour is dispatched through the customer objects
Deleted	none	–

Stage 2 → Stage 3 (compare stage-2...stage-3)

Change	Files	Reason
New	Host, ComboMeal, PolicyManager, Stage3Checks, Stage3IntegrationChecks, docs/IMPLEMENTATION_NOTES.md	The new role, menu item and pricing policy
Changed	Customer (+8 lines)	Adds recoverSatisfaction() for Happy Hour, capped at 100
Changed	Order (+14 lines)	Adds applyDiscount(), allowed only while IN_PROGRESS
Changed	GameController (+106), GameCLI (+59)	Host phase, Happy Hour activation, recovery and timer, order-time discount; Host and Happy Hour prompts and displays
Changed	Main (+3)	Registers the combo, the Host and the PolicyManager (the setup change the brief expects)
Changed	ArrivalSchedule (6 lines)	Validation rewritten as an ordinary loop; no behaviour change
Deleted	none	–

The Stage 1 and Stage 2 checks still pass on Stage 3, which shows the earlier behaviour still works.

After the tags: a behaviour-preserving refactor (compare stage-3...main)

After tagging, Minh reviewed the code against SOLID and the team agreed three changes. Every check and both replays gave identical results after each one, and the stage tags still show each stage as delivered.

Change	Files	Why
Template method for billing	Customer (calculateBill() final, new applyPricing()), the three subtypes	The price check happens once; a new customer type cannot forget it
GameStats takes over counting	new GameStats; GameController	Single responsibility: the controller coordinates, GameStats counts and decides victory

seat(), abandon(), pay() replace setState()	Customer, Waiter, Host, Cashier, GameController	Stronger encapsulation: Customer checks each step itself
Happy Hour usage recorded only in PolicyManager	PolicyManager, GameController	Removes duplicated state

7. Functional evidence

The final code passes every check suite, and the two checked-in replays end in VICTORY and DEFEAT exactly as the README states. All results below were rerun on main (1ec828e) with --release 17.

Check suites

Suite	Result	What it covers
DomainChecks	105 passed, 0 failed	Domain and staff rules: one task per turn, guarded order and customer transitions, queue and table invariants, satisfaction bounds, Cashier selection
IntegrationChecks	74 passed	Full games through the real controller and CLI, timing scenarios, thresholds, input retry, both replays
Stage2Checks	17 passed, 0 failed	VIP and Critic loss rates, VIP pricing, most-expensive lookup and its tie rule
Stage2IntegrationChecks	passed	Six arrivals with a VIP and a Critic; polymorphic VIP bill and Critic choice through the controller
Stage3Checks	41 passed, 0 failed	Host, ComboMeal ($24.30, 3 units), Happy Hour timer and discount (VIP Burger $10.80)
Stage3IntegrationChecks	31 passed	Host and Happy Hour inside real turns

The README also reports that the current build matched an independent rules model across 72 full games, 1,296 turns and 15,857 rejected inputs.

Victory replay (examples/victory.txt): VICTORY, 5 paid, $73.80 revenue, 84.80 average, 18 turns. The manager seats every arrival straight away, keeps the Chef cooking, serves each dish the turn it is ready, and switches on Happy Hour on turn 5.

Customer	Type	Dish (customer's choice)	Bill	Paid on turn	Satisfaction recorded
#1	Regular	Salad	$9.00	4	92
#2	VIP	Burger	$13.50	7	100
#3	Regular	Pasta	$18.00	11	84

#4	Critic	Salad + Pasta Combo	$24.30	15	64
#5	Regular	Salad	$9.00	17	84
#6	Regular	Burger	not paid	–	arrives turn 16, too late (as the brief intends)

Defeat replay (examples/defeat.txt): DEFEAT, 0 paid, $0.00 revenue, average N/A, 18 turns. The manager declines Happy Hour and answers wait to every staff prompt (72 inputs). Nobody is seated, Regular #1 leaves unhappy on turn 13 and Critic #4 on turn 18, and the other four are still queued at the end.

Scenario checks that match the brief

- The brief's Pasta example: seated turn 1, ordered and first unit cooked turn 2, READY turn 3 (the Waiter cannot serve it), served turn 4, ready to pay turn 5, paid turn 6 at satisfaction 76.
- Abandonment: a seated customer whose order is never finished reaches 0 and leaves; the order is CANCELLED, the table freed, no revenue added, and the departure counted once.
- Turn phases: no action before beginTurn(), no duplicate arrivals, no Chef action before the Waiter's phase ends, one Waiter task per turn.
- Thresholds: exactly 4 paid, $45.00 and an average of 60 wins; one short on any target loses.
- Invalid input: nonsense, bad IDs, ordering twice and serving early are explained and retried without using the task; end of input stops the session without declaring a result.
- Stage 2: the VIP pays $13.50 for a Burger through the controller; the Critic orders Pasta on the base menu.
- Stage 3: the Host seats the queue head at the lowest free table, at most once a turn, without using the Waiter's task; Happy Hour recovers exactly 15 on activation and on its second turn, never a third time, never for eating, paying or departed customers; an order placed before Happy Hour keeps its locked bill.

Figures 3–5 are in Appendix E at the end of the report: the end of the victory replay, the end of the defeat replay, and an "Invalid choice … Try again." retry in a live game.

8. Reflections

8.1 Group reflection

What worked. Agreeing an exact API contract before coding (CODING_PLAN.md) let the domain and controller code be written in parallel and integrated stage by stage. Tagging a stage only after its real checks passed gave us three snapshots that each run on their own. Stage 2 then showed the design paying off: two new customer types needed no change to the controller or CLI.

What was hard.

- The Host changed late. Our design had an automatic Host; we switched to a player-controlled one after Stage 3 was first integrated, so the controller, CLI, replay files and diagrams all had to be updated together.
- Keeping the diagrams, the README and the code in step took deliberate effort, especially after the refactor, which needed its own diagram and documentation pull requests.

What we would do differently.

- Settle player-facing behaviour, such as whether the Host is automatic, before coding the stage.
- Have every member commit their own work from their own account.
- Update diagrams and replay files in the same pull request as the code change they describe.
- If more staff roles were expected, introduce a common staff-action abstraction instead of adding a phase per role.

8.2 Individual reflections: pointers for each member

Devansh (controller, CLI, setup, integration)

My main contribution was putting the game together across the three stages. I worked on GameCLI and GameController, connected them to Zheng Ning's customer and staff classes, and checked that each stage could run on its own.

One thing I learned about OOP was how to decide which class should be responsible for a rule. For example, when I type serve 2 at an empty table in the demo, the CLI reads the command and displays the error, while the controller checks whether the action is allowed. I can then type serve 1 without losing the Waiter's action. That example helped me understand why the same class shouldn't handle both the terminal interaction and the game rules.

The hardest part was getting the timing right. A dish can become ready when the Chef cooks it, but it can't be served immediately because the Waiter has already had their turn. I separated the turn into phases and checked the full path from cooking to payment, including which turn each event happens on. The demo and the integration checks show that sequence.

If I did the project again, I'd write out a few examples turn by turn before coding the controller. Having the expected state after each turn written down would have made it easier to spot timing mistakes when we integrated the classes.

Zheng Ning (domain and staff classes)

In this project I was responsible for the domain and staff classes. I wrote 19 classes in total: Customer and its three subtypes (RegularCustomer, VIPCustomer, CriticCustomer), CustomerState, Staff and its four subtypes (Waiter, Chef, Cashier, Host), Order, OrderStatus, Menu, MenuItem, ComboMeal, Table, WaitingQueue, PolicyManager and InvalidActionException. I also wrote their checks in DomainChecks, Stage2Checks and Stage3Checks. Devansh committed my code to GitHub when he integrated it, so my work is credited in docs/IMPLEMENTATION_NOTES.md and docs/STAGE_CHANGES.md.

The OOP idea I learned the most from is polymorphism. In it each customer type overrides its own satisfaction loss rate, pricing and food choice. Because of this, the controller treats every customer the same way and never checks what type it is. This paid off in Stage 2 where we added the VIP and Critic customers without changing GameController or GameCLI at all.

The hardest part for me was blocking illegal actions, such as serving a dish that is not ready yet, without breaking the game. I solved this by making every staff method follow the same order. It first checks every rule, then changes the game state, and only marks the staff member as having acted at the very end. This way, a wrong command leaves the game exactly as it was, and the player can simply try again without losing that staff member's turn.

Braden (class and sequence diagrams)

My main role was the UML design. I drew the class diagrams for all three stages and the sequence diagram and kept them matched to the code that Devansh, Zheng Ning and Minh wrote. I drew Stage 1 first as the base and made sure Stage 2 and Stage 3 only add to it. These files and SC2002_Sequence_Diagram.drawio are in the commit of 8 Nov 2026.

The OOP idea I learned the most from is the difference between composition, aggregation and association, and how each one shows up in the code. GameController's parts, such as the tables, the waiting queue and the four staff members, are private final fields. Main builds them once and passes them in, so they live and die with the controller, and I drew them with filled diamonds. A reference that can be empty is only an association: a table can be free, so its current customer is 0..1. Customers pass through the WaitingQueue on their way to a table, so the queue holds them without owning them. To decide each line, I had to check the field, its multiplicity and who creates the object, not just guess from the class names.

The hardest part was making the diagrams match the code exactly: signatures, visibility, multiplicities and static members. Devansh's controller and Zheng Ning's classes kept changing, so I checked each class against its source file. That is how I added GameController.finishWaiterPhase(), marked MAX_TURNS as public static final and made happyHourActive a derived attribute. For the sequence diagram, I wanted a scenario where one manager choice changes the result. I followed one Regular customer through four turns. With Happy Hour in turn 2, the satisfaction total is 247 and the game is won. Without it, the total is 239, which misses the target of 240. I then checked that every message in it is a method that exists in the class diagram.

If I were to do another project, I would draw from the code earlier and update the diagrams in the same pull request as each code change. Minh did this during his refactor, and it was much easier than catching up afterwards.

Minh (documentation, SOLID review, refactor)

My main role was the documentation: the README, the stage notes in docs/, this report and the video plan. While writing them I had to read the whole code base, so after the stage tags I also reviewed it against the SOLID principles. I proposed the changes to the group, made them once they agreed, and merged them through pull requests #2 to #6. The commits that add the template method for billing, GameStats and seat()/abandon()/pay() are mine.

The OOP idea I learned most from is the single responsibility principle. GameController used to run the turns and also keep the score: five counters added up in several places, plus the victory rule at the bottom. I moved the counting and the three targets into GameStats, so the controller only coordinates turns and each class has one reason to change. Seeing the before and after in the same file made the principle concrete for me in a way the lecture example had not.

The hardest part was changing a working design without breaking anything, including the diagrams. I handled it by making one change per commit and running every check and both replays after each one; the victory replay had to give exactly 5 paid, $73.80 and $84.80 every time. I also updated the class and sequence diagrams in the same pull request as the code, and fixed the arrows when the Customer box grew.

If I did it again, I would review the code against SOLID before each stage tag rather than after the last one. Then GameStats would have existed from Stage 1, and the refactor would not have needed separate diagram and documentation updates.

Appendix

A. Video segments and timestamps

#	Segment required by the brief	Start time	Presenter
1	The game launching from the submitted project	4.06	Devansh
2	Meaningful manager choices, including a decision where staff are constrained	5.37	Devansh
3	A dish going READY → SERVED → payment	4.35	Devansh
4	A VIP or Critic behaviour (VIP 10% pricing discount)	5.15	Devansh
5	A Stage 3 feature	4.22	Devansh
6	One victory and one defeat	9.37(V), 10.00(D)	Zheng Ning
7	The stage tag comparisons	1.19	Minh
8	Diagrams	12.00	Braden

Video link: https://www.youtube.com/watch?v=y8gwKpnKHsA

B. Commands

```
git clone https://github.com/DevanshJ07/sc2002-restaurant-rush.git
cd sc2002-restaurant-rush
powershell -ExecutionPolicy Bypass -File scripts/run.ps1
powershell -ExecutionPolicy Bypass -File scripts/build.ps1 -Test
```

```
Get-Content examples/victory.txt | java -cp out restaurantrush.Main
Get-Content examples/defeat.txt | java -cp out restaurantrush.Main
```

On macOS or Linux: sh build-and-check.sh, then java -cp out restaurantrush.Main < examples/victory.txt.

C. Submission checklist

One archive per group, named LABGroup_AssignmentGrp.zip (for example FCS1_Grp1.zip).

- ☐ Source code with build and run instructions (the repository at main)
- ☐ README with the repository URL, stage comparisons and video timestamps
- ☐ This report as a PDF, with Figures 1–5 inserted and the bracketed placeholders filled in
- ☐ The recorded video, or a download link that stays available
- ☐ Signed Declaration of Original Work
- ☐ Each member's Declaration on Use of GAI Assistance

D. References

- SC2002 Restaurant Rush group assignment brief (course document).
- Restaurant Rush repository: source, README, CODING_PLAN.md, docs/IMPLEMENTATION_NOTES.md, docs/STAGE_CHANGES.md, diagrams.

E. Screenshots

Figure 3: End of the victory replay (examples/victory.txt): VICTORY with 5 paid customers, $73.80 revenue and an average satisfaction of 84.80.

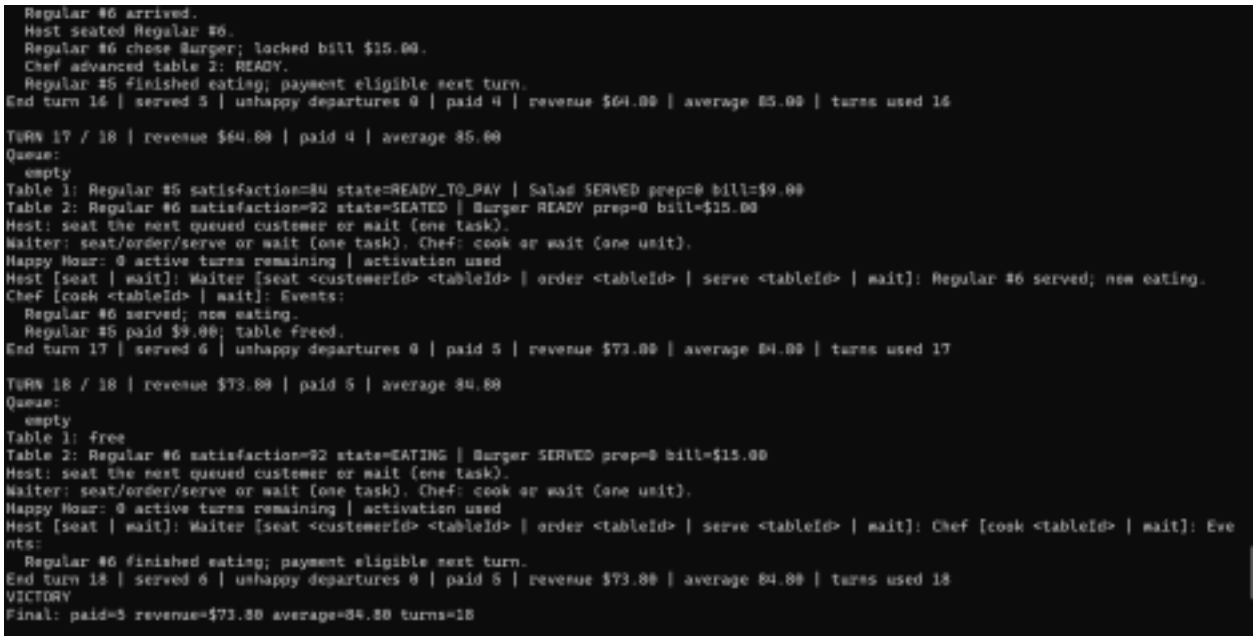


Figure 4: End of the defeat replay (examples/defeat.txt): Critic #4 leaves unhappy on turn 18, and the game ends in DEFEAT with 0 paid, $0.00 and average N/A.

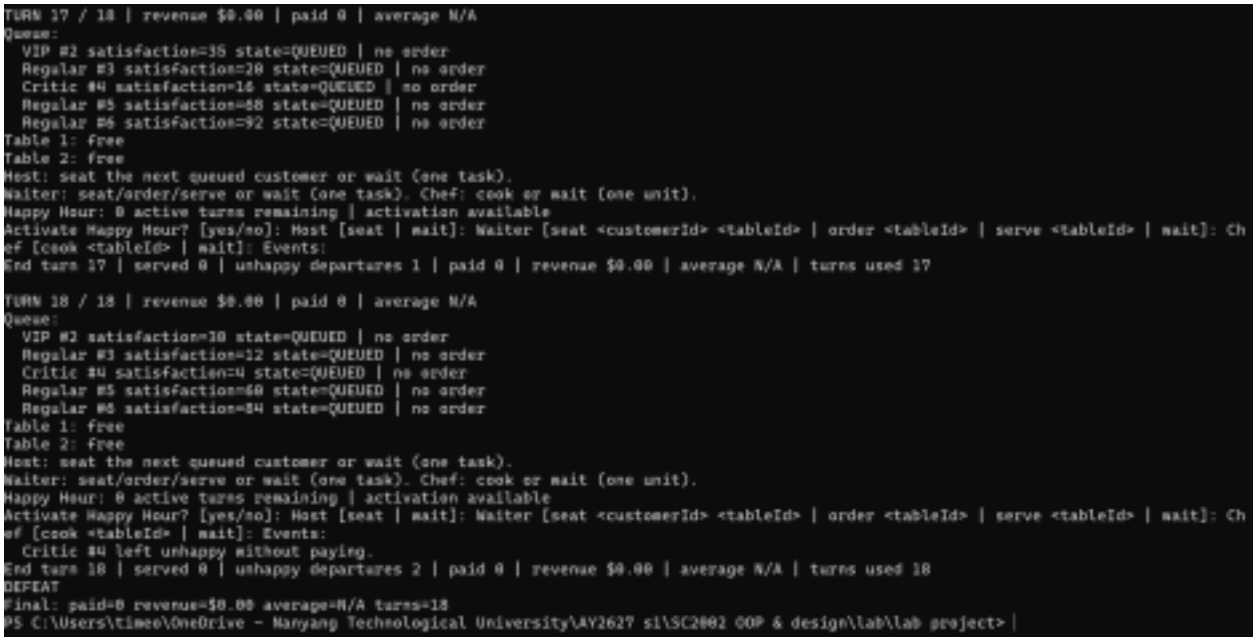


Figure 5: Invalid input in a live game. “serve 2” is refused with “Invalid choice: Table 2 is empty. Try again.” and the Waiter prompt returns without using the Waiter’s task.

